UWE Bristol | University of the West of England

Presentation by

Majid Mumtaz

Lecture 9: More on GUI using Python

Week 9

Overview

UWE Bristol University of the West of England

Recap

Examples

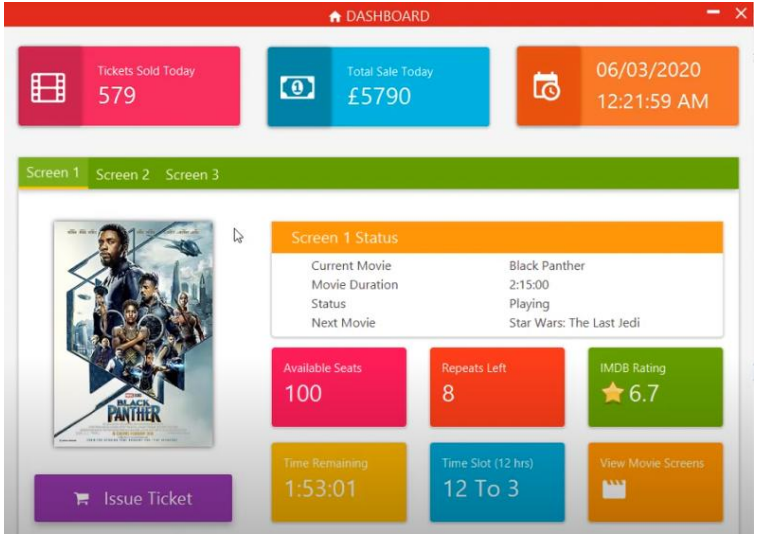
Event-Driven Programming

Widgets - More on GUI with Python

OOP & MVC way to write Tkinter applications

Recap

- Human Computer Interaction ➡ Graphical User Interface (GUI)
- Design Principles
- GUI Programming with Python
- Tkinter and Custom Tkinter Examples



Dashboard of Movie Ticket booking: https://www.youtube.com/watch?v=CrQ3Fr306pM

Recap - Tkinter Widgets

Entry
Text
Scrollbar
Checkbox
Radio Button
Combobox
Listbox
Menu
Separator

Frame
LabelFrame
Notebook
Treeview

Many others… e.g., Slider, Spinbox, Sizegrip, Progressbar, Canvas, Cursors

Recap (GUI with Tkinter)

UWE Bristol University of the West of England

Tkinter important component is the **layout manager**, which helps shape the widgets within a parent window.

Three Core Classes are:

1. **pack()** method organizes the widgets in blocks before placing in the parent widget.

2. **grid()** method organizes the widgets in grid (table-like structure) before placing in the parent widget.

3. **place()** method organizes the widgets by placing them on specific positions directed by the programmer.

GUI with Tkinter - place()

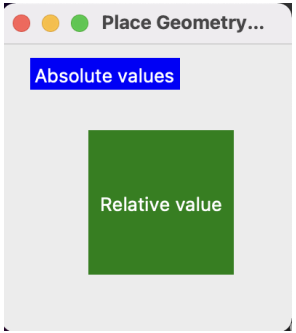
UWE Bristol University of the West of England

- Rarely used because changing on widget place will require adjusting all other widgets…
- Useful when end-user would like to place one/more specific widget(s) on specific place on the application window

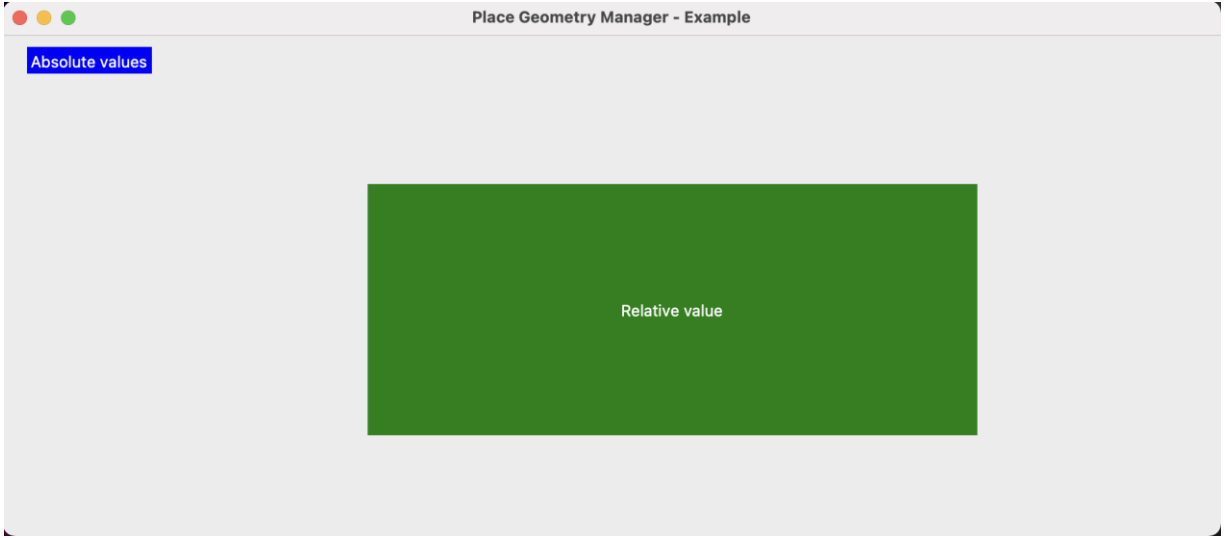
- widget.place(**options) – *x, y, relx, rely, width, height, relwidth, relheight, anchor*
- *x,y are for absolute positions and relx,rely are for relative positions*
- *width,height for absolute values and relwidth,relheight* for *relative values (floating point b/w 0.0 and 1.0* - represent a fraction of the w/h of the container*)*
- *anchor – N, E, S, W, NW, SE, or SW – default NW*

GUI with Tkinter - place()

```python
import tkinter as tk

root = tk.Tk()
root.title('Place Geometry Manager - Example')

label1 = tk.Label(root, text="Absolute values", bg='blue', fg='white').place(x=20, y=10)
label2 = tk.Label(root, text="Relative value", bg='green', fg='white').place(relx=0.8, rely=0.8, relwidth=0.5, relheight=0.5, anchor='se')

root.mainloop()
```



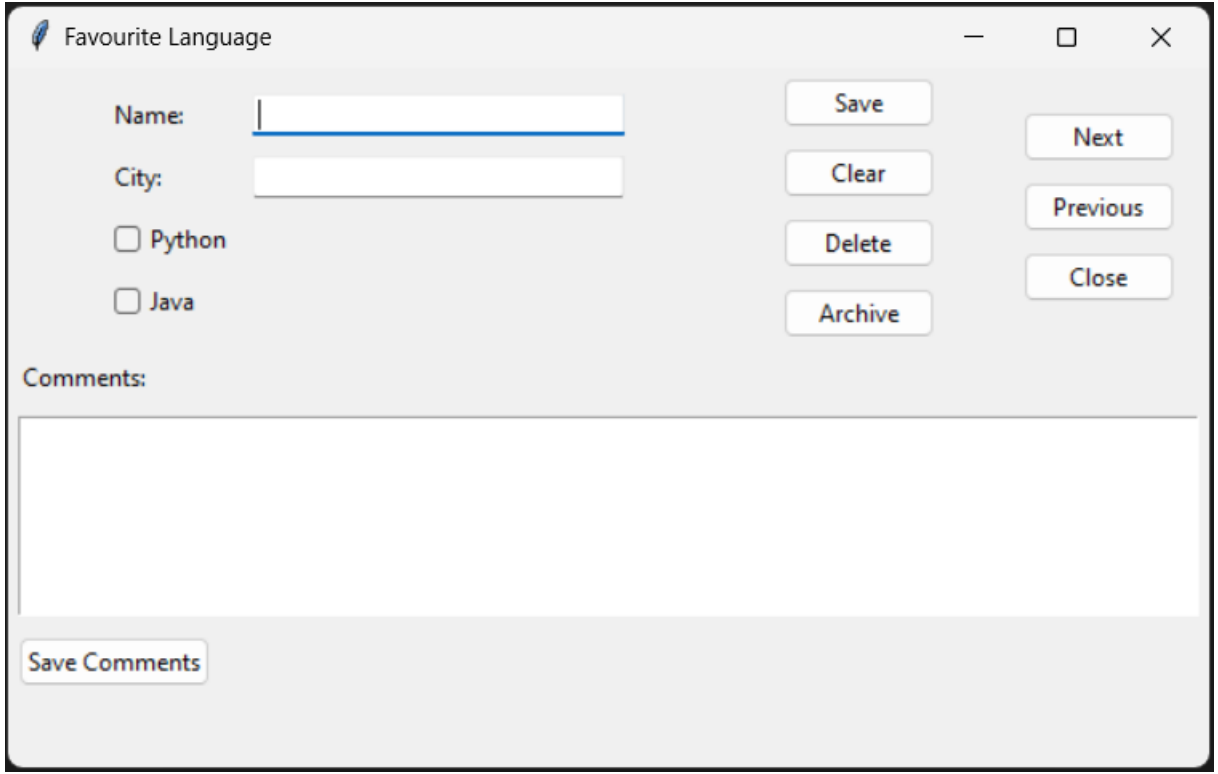
1c_placegeometry.py



Demo

Read: https://www.pythontutorial.net/tkinter/tkinter-place/

Exercise: Can you design a GUI in Tkinter as shown in the image?



Homework?

Tkinter Widgets

UWE Bristol University of the West of England

Frame
Text
Scrollbar
Checkbox
Radio Button
Listbox
Combobox
Menu
Separator
TreeView
Notebook

- Displays a simple rectangle and is very useful to organize widgets both visually and programmatically
- *frame = ttk.Frame(container, **options)*
- **options can be height, width, borderwidth, padding, relief, etc.
- Demonstrate frame example

A Simple Frame exan

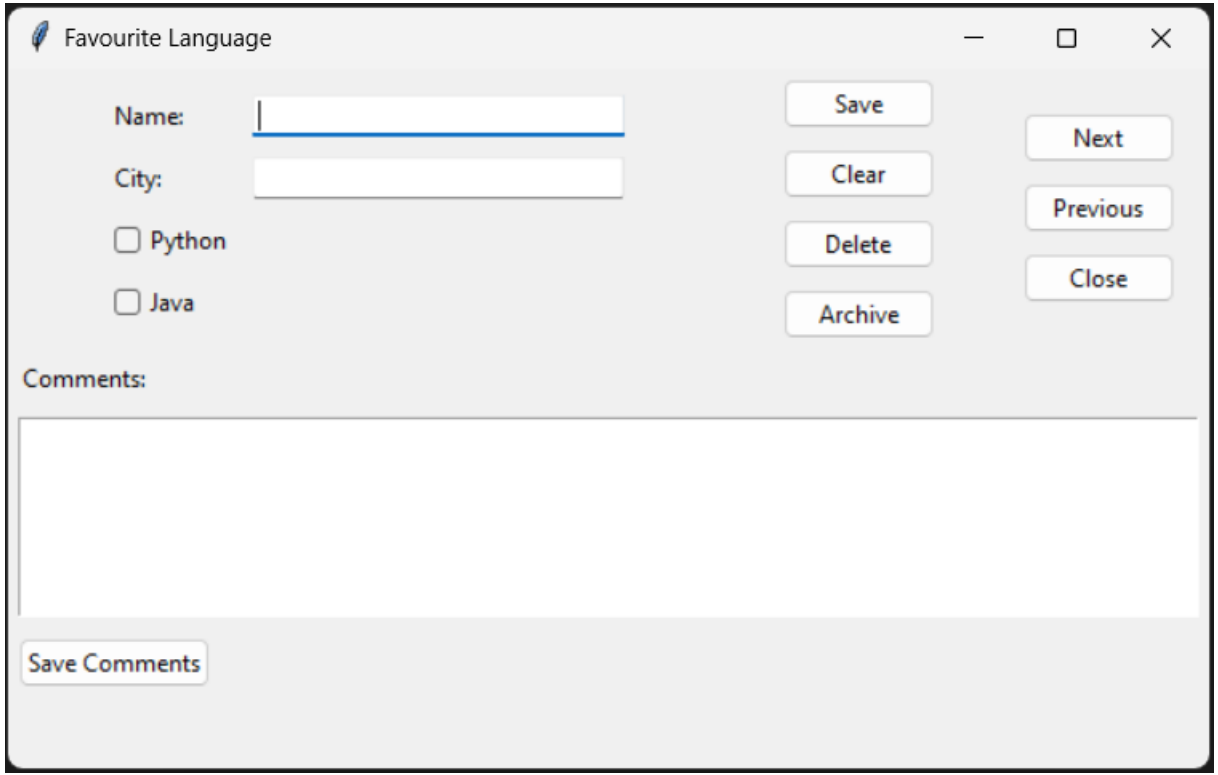
ASD > ASD > Lab8 > BB > Lecture examples > frameexample_4.py > ...

```python
# A simple frame, username text field and search button that di
# what user has entered through a message box
from tkinter import *
import tkinter.messagebox as mb

def validate():
    print("Username entered :", user.get())
    mb.showinfo(title='Information', message='OOP Tkinter! Hell

w = Tk()
w.geometry("200x70")
w.title("My App")

f1 = Frame(w)
f1.pack(side=TOP)
u_label = Label(f1,text="Username")
u_label.pack(side=LEFT)
user = StringVar()
u_entry = Entry(f1, textvariable= user, bd=2)
u_entry.pack(side=RIGHT)
u_button = Button(text="SEARCH", command=validate)
u_button.pack(side=BOTTOM)

w.mainloop()
```

ASD > ASD > Lab8 > BB > Lecture examples > frameexample_4_OOP_v1_1.py > MainFrame

```python
# OOP version to demonstrate Frames with grid layout
# A simple username text field and search button that displays
# what user has entered through a message box

import tkinter as tk
from tkinter import ttk
import tkinter.messagebox as mb

class App(tk.Tk):
    def __int__(self):
        super().__init__()
        self.title('My OOP App')
        self.geometry('200x70')

class MainFrame(ttk.Frame):
    def __init__(self, container):
        super().__init__(container)
        options = {'padx': 5, 'pady': 5} # padding helps to show widgets fully
        #options will be passed as parameter to pack()

        self.u_label = ttk.Label(text="Username")
        self.u_label.grid(column=0, row=0, **options)
        self.user = tk.StringVar()
        self.u_entry = ttk.Entry(textvariable=self.user)
        self.u_entry.grid(column=1, row=0, **options)
        self.u_button = ttk.Button(text="SEARCH", command=self.validate)
        self.u_button.grid(column=0, columnspan=2, row=1, **options)

    def validate(self):
        print("Username entered :", self.user.get())
        mb.showinfo(title='Information', message='OOP Tkinter! Hello ' + self.user.get())

if __name__ == "__main__":
    app = App()
    frame = MainFrame(app)
    app.mainloop()
```

Can you design a GUI in Tkinter using Frames?

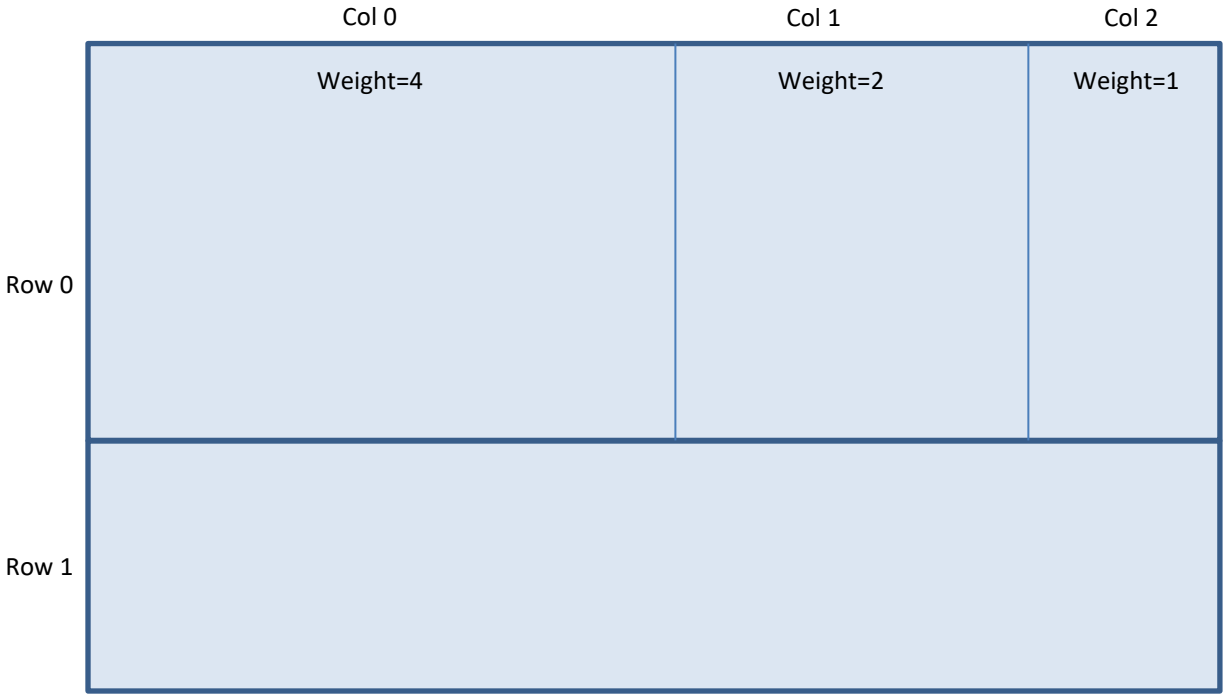


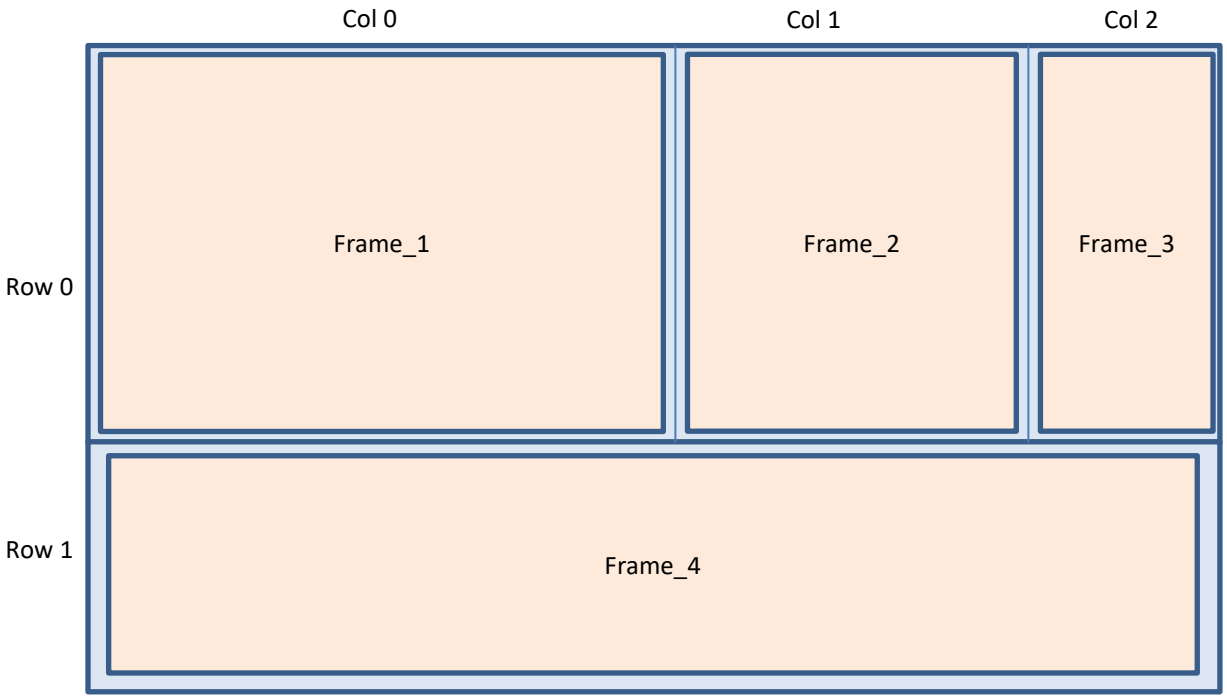
Exercise

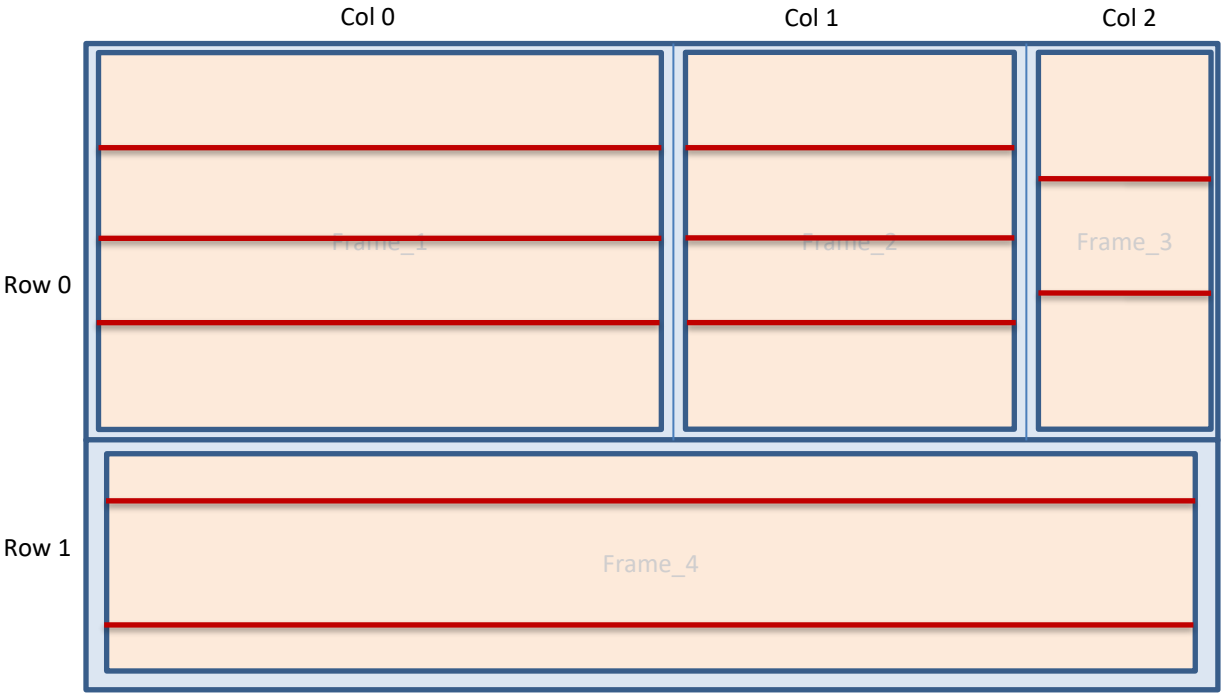
GUI Design: Grid geometry manager, 2 rows and 3 columns

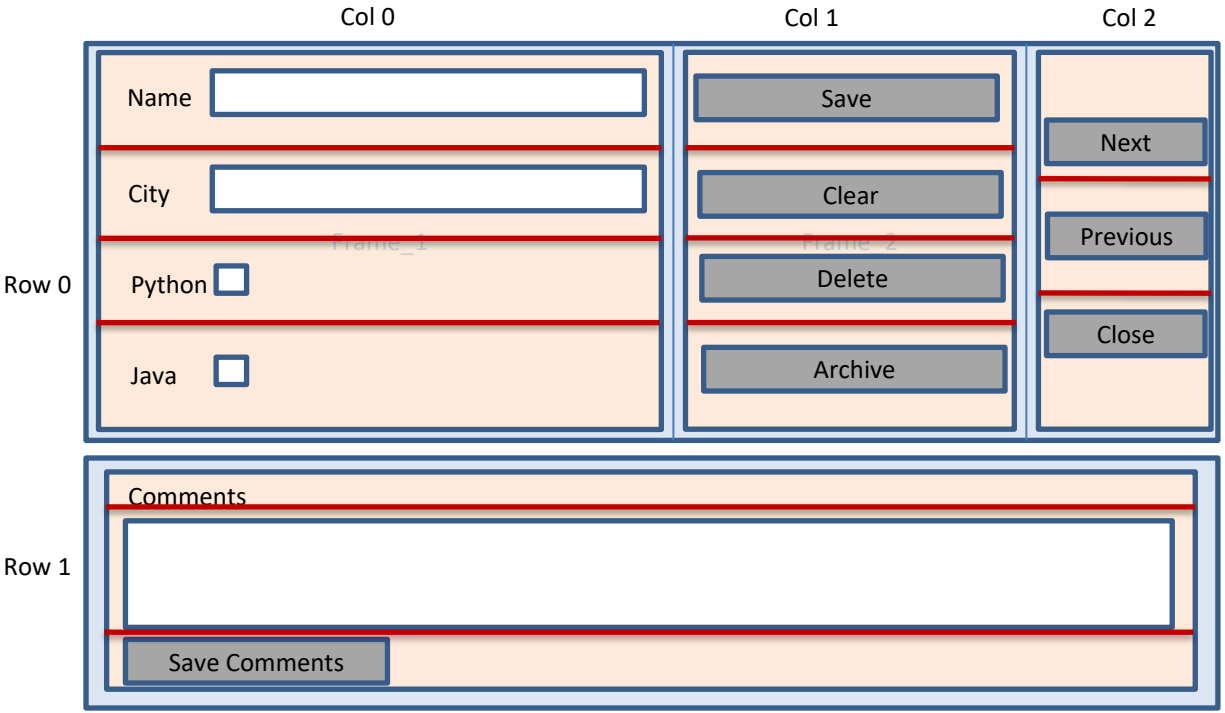
```
# Row 0 has three frames.
# Frame 1 in Column 0, Frame 2 in Column 1, Frame 3 in Column 2
# -- Frame 1:
#         -- has labels and fields for Name and City.
#         -- has two check boxes: Python and Java
# -- Frame 2:
#         -- has four buttons
# -- Frame 3:
#         -- has three buttons
#
# Row 1 has one frame Frame 4.
# -- Frame 4:
#         -- has one label, one text field and one button
```

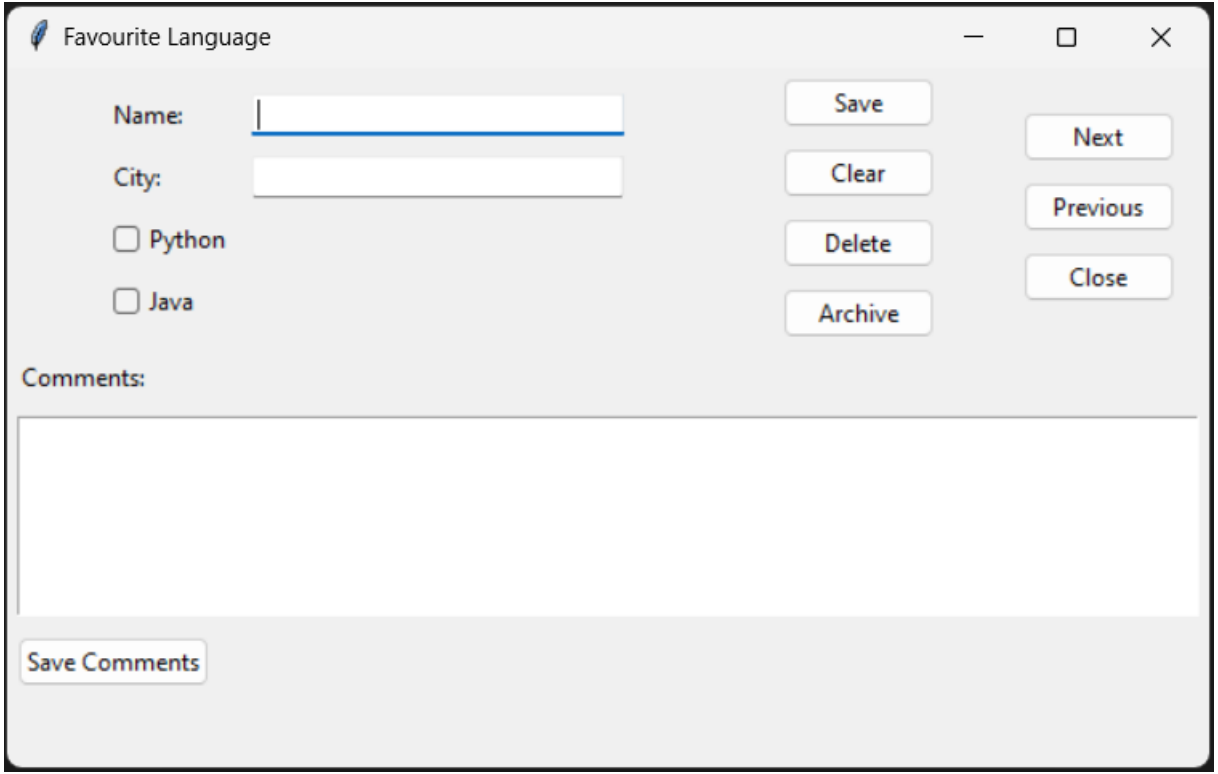
Main window


Col 0
Col 1
Col 2
Weight=4
Weight=2
Weight=1
Row 0
Row 1


Col 0
Col 1
Col 2
Row 0
Row 1
Frame_1
Frame_2
Frame_3
Frame_4


Col 0
Col 1
Col 2
Row 0
Row 1
Frame_1
Frame_2
Frame_3
Frame_4


Col 0
Col 1
Col 2
Row 0
Row 1
Name
City
Python
Java
Save
Clear
Delete
Archive
Next
Previous
Close
Comments
Save Comments



3_framewidget_homework.py

ASD > Examples > Widgets > framewidget.py > create_frame_1

```python
import tkinter as tk
from tkinter import ttk, TclError, Text

def create_frame_1(container):
    frame1 = ttk.Frame(container)

    frame1.columnconfigure(0, weight=1)
    frame1.columnconfigure(1, weight=3)

    ttk.Label(frame1, text='Name:').grid(column=0, row=0, sticky=tk.W)
    name_entry = ttk.Entry(frame1, width=30)
    name_entry.focus()
    name_entry.grid(column=1, row=0, sticky=tk.W)

    ttk.Label(frame1, text='City:').grid(column=0, row=1, sticky=tk.W)
    city_entry = ttk.Entry(frame1, width=30)
    city_entry.grid(column=1, row=1, sticky=tk.W)

    pythonlanguage = tk.StringVar()
    pythonlanguage_check = ttk.Checkbutton(
        frame1,
        text='Python',
        variable=pythonlanguage,
        command=lambda: print(pythonlanguage.get()))
    pythonlanguage_check.grid(column=0, row=2, sticky=tk.W)

    javalanguage = tk.StringVar()
    javalanguage_check = ttk.Checkbutton(
        frame1,
        variable=javalanguage,
        text='Java',
        command=lambda: print(javalanguage.get()))
    javalanguage_check.grid(column=0, row=3, sticky=tk.W)

    for widget in frame1.winfo_children():
        widget.grid(padx=5, pady=5)

    return frame1
```

ASD > Examples > Widgets > framewidget.py > create_frame_1

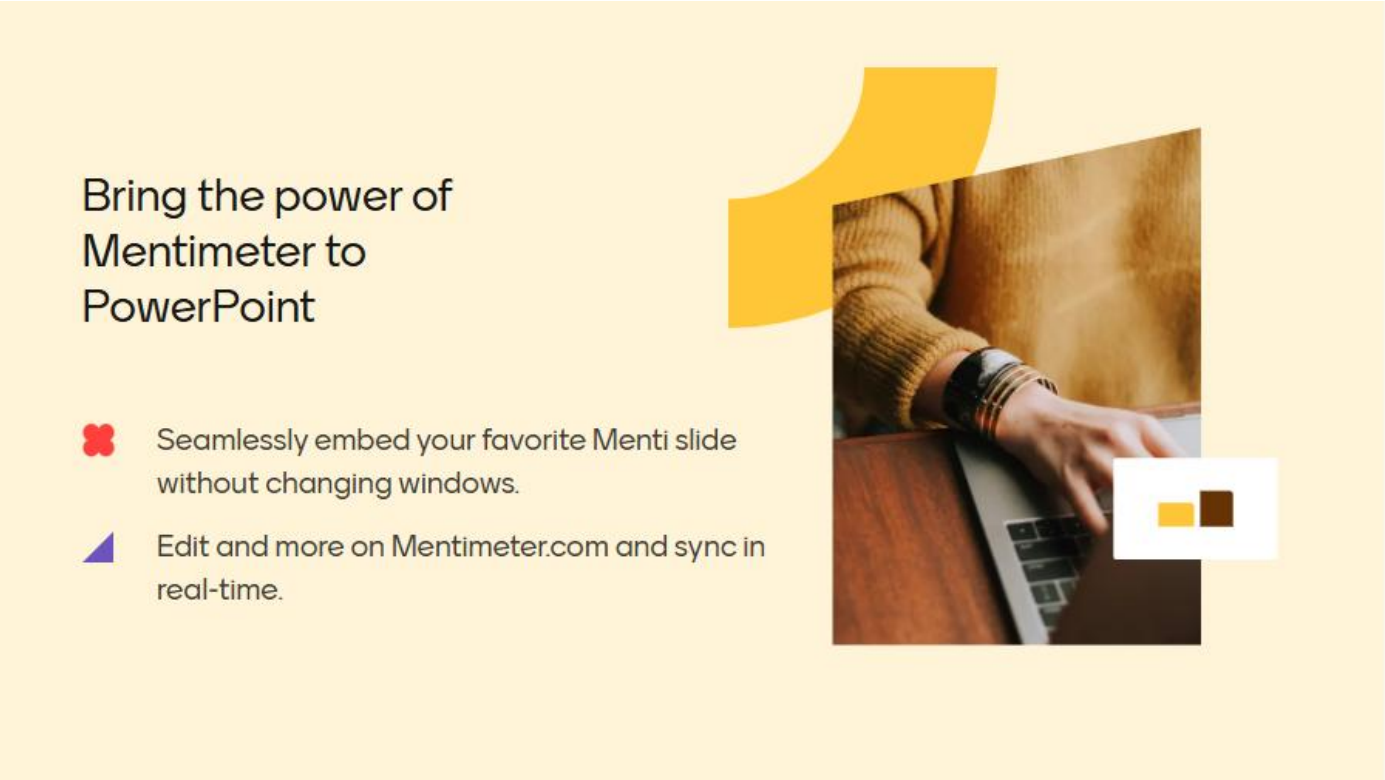
```python
def create_frame_2(container):
    frame2 = ttk.Frame(container)

    frame2.columnconfigure(0, weight=2)

    ttk.Button(frame2, text='Save').grid(column=0, row=0)
    ttk.Button(frame2, text='Clear').grid(column=0, row=1)
    ttk.Button(frame2, text='Delete').grid(column=0, row=2)
    ttk.Button(frame2, text='Archive').grid(column=0, row=3)

    for widget in frame2.winfo_children():
        widget.grid(padx=5, pady=5)

    return frame2

def create_frame_3(container):
    frame3 = ttk.Frame(container)

    frame3.columnconfigure(0, weight=1)

    ttk.Button(frame3, text='Next').grid(column=0, row=0)
    ttk.Button(frame3, text='Previous').grid(column=0, row=1)
    ttk.Button(frame3, text='Close').grid(column=0, row=2)

    for widget in frame3.winfo_children():
        widget.grid(padx=5, pady=5)

    return frame3
```

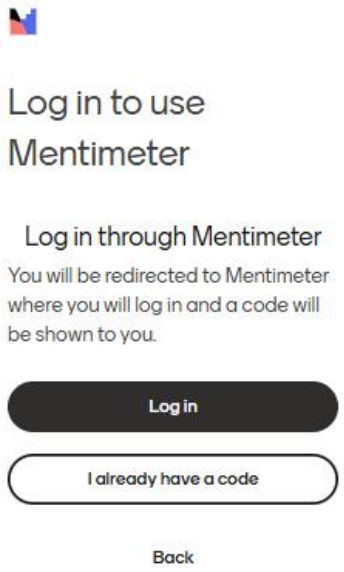
ASD > Examples > Widgets > framewidget.py > ...

```python
def create_frame_4(container):
    frame4 = ttk.Frame(container)

    frame4.columnconfigure(0, weight=7)

    ttk.Label(frame4, text='Comments:').grid(row=0, sticky=tk.W)
    tk.Text(frame4, height=6).grid(row=1)
    ttk.Button(frame4, text='Save Comments').grid(row=2, sticky=tk.W)

    for widget in frame4.winfo_children():
        widget.grid(padx=5, pady=5)

    return frame4


def create_main_window():
    root = tk.Tk()
    root.title('Frame Example')
    root.resizable(0,0)

    try:
        root.attributes('-toolwindow', True)
    except TclError:
        print(' - toolwindow Not supported on your platform')

    root.columnconfigure(0, weight=4)
    root.columnconfigure(1, weight=2)
    root.columnconfigure(2, weight=1)

    frame_1 = create_frame_1(root)
    frame_1.grid(column=0, row=0)

    frame_2 = create_frame_2(root)
    frame_2.grid(column=1, row=0)

    frame_3 = create_frame_3(root)
    frame_3.grid(column=2, row=0)

    frame_4 = create_frame_4(root)
    frame_4.grid(row=1, columnspan=3)

    root.mainloop()

if __name__ == "__main__":
    create_main_window()
```

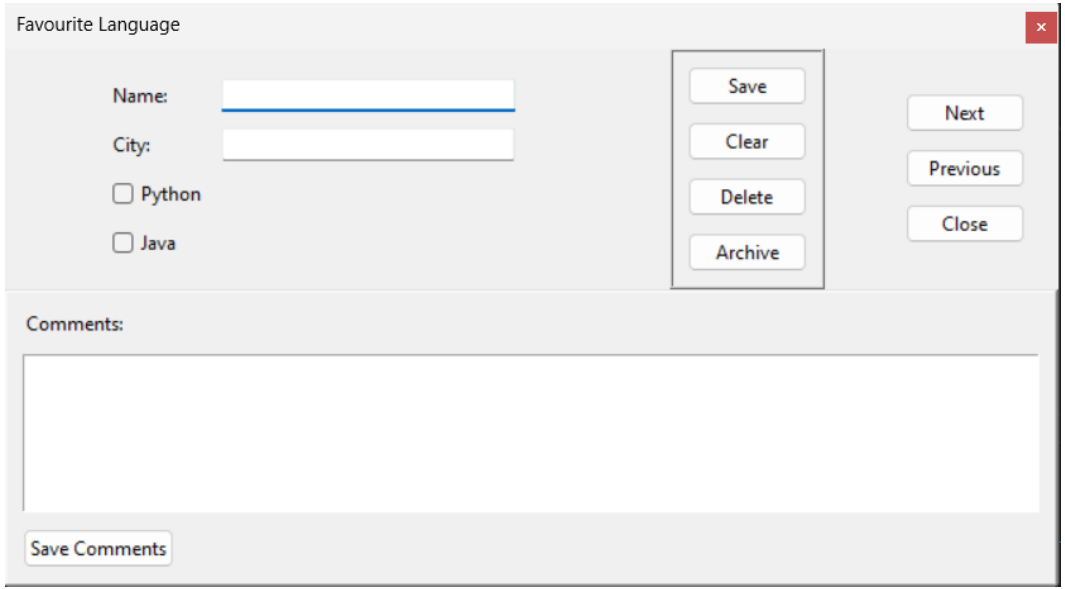
Demo

Now you know how to design and implement GUIs using Tkinter frame widget.





Now you know how to design and implement GUIs using Tkinter frame widget. Now you should be able to apply other changes e.g., add borders around frames.



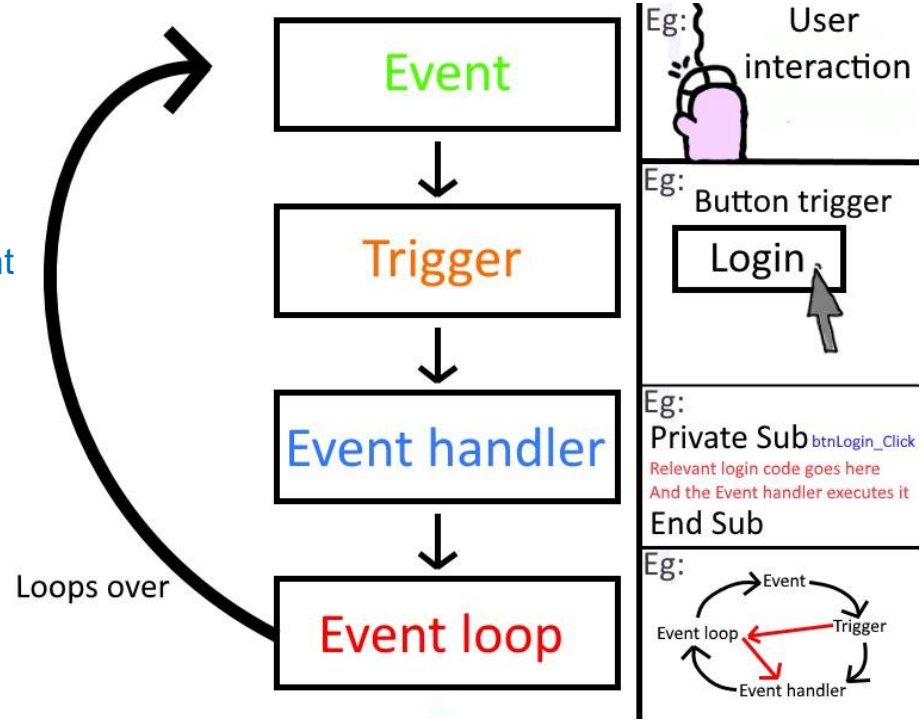
Check borderwidth and relief properties when instantiating a Frame.

```
frame2 = ttk.Frame(container, borderwidth=1, relief=RIDGE, padding=5)
frame4 = ttk.Frame(container, borderwidth=2, relief=RAISED, padding=5)
```

Event Driven Programming

An **event** is triggered, and then handled by taking some **action**

The main loop of event – driven programming



Special cases
GUI development
Exception handling

Event Driven Programming

UWE Bristol University of the West of England

GUI development is also a special case of programming practices, where actions are taken on event happenings.

widgets have some associated behaviours, e.g.,

when a button is **clicked**,

text is **filled** into a text field, (focusIn, focusOut etc.)

an option is **selected** from a list-box.

These types of user behaviours are called ***events,***

Actions that the GUI takes to respond to such events are known as ***callbacks.***

Event Handling (command attribute)

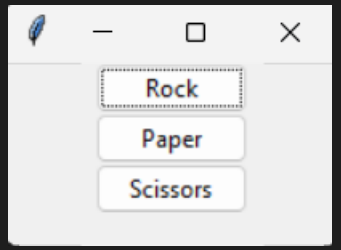
```
from tkinter import *

def validate():
    print("Username entered :", user.get())

#window
w = Tk()
w.geometry('300x100')
w.title('Info Form')

#user label and text entry box
userLabel = Label(w, text="User Name").grid(row=0, column=0)
user = StringVar()
userEntry = Entry(w, textvariable=user).grid(row=0, column=1)

loginButton = Button(w, text="Login", command=validate).grid(row=4, column=0)

w.mainloop()
```

definition before use

no brackets

Event Handling (command attribute)

```python
import tkinter as tk
from tkinter import ttk
from random import randint

root = tk.Tk()

# callback function with parameters
def selectwinner(options):
    givenoptions = ['Rock', 'Paper', 'Scissors']
    selection_index = randint(0,2)
    if givenoptions[selection_index] == options:
        print('Well done! ', options, ' Won!')
    else:
        print('Better luck next time! ', options)

# Use lambda function if you want to pass parameters to command binding callback function
ttk.Button(root, text='Rock', command=lambda: selectwinner('Rock')).pack()
ttk.Button(root, text='Paper',command=lambda: selectwinner('Paper')).pack()
ttk.Button(root, text='Scissors', command=lambda: selectwinner('Scissors')).pack()

root.mainloop()
```




```
Well done!  Rock  Won!
Better luck next time!  Paper
Better luck next time!  Scissors
Better luck next time!  Paper
Better luck next time!  Scissors
Well done!  Paper  Won!
```

Demo

Command option isn't available in all widgets; binds left-click and the backspace only (not Return key)… to overcome command limitations event binding is used…

Event Handling

UWE Bristol University of the West of England

Event handling in `tkinter` is conducted in two ways:

1) Using **`command`** attribute of the widget to call the *callback* function specially developed

Following Widgets have command attribute:

`Button, Checkbutton, Radiobutton, Scale` etc.

`Menu, Entry, Combobox` do not have that option, but they use other properties to respond to user events

2) Use the **`bind`** function to deal with the events at a lower level.
The triggers for binding actions include mouse move, selection, keypress etc.

Event Binding

UWE Bristol University of the West of England

Event binding: Assigning a function to an event of a widget

widget.bind(event, handler, add=None)

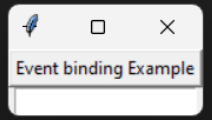
When an event occurs in the widget, tkinter will automatically call the handler with the event details

To register additional handler, you can pass '+' to the add argument allowing you to have multiple event handlers to respond to the same event

Event binding example

5a_eventbinding_v0.py > ...

```python
import tkinter as tk
from tkinter import ttk
counter =0
def my_event_callback(event):
    global counter
    print('The process event fucntion was called ', counter)
    counter = counter + 1

window = tk.Tk()
window.geometry("+900+250")
my_btn = tk.Button(window, text="Event binding Example")
my_entry = tk.Entry(window)

my_btn.bind("<Enter>", my_event_callback) # This binds the "Enter" event (when mouse hovers over)

my_entry.bind("a", my_event_callback) # This binds key "a" event

my_btn.pack(expand=True)
my_entry.pack(expand=True)

window.mainloop()
```



Demo

Output

```
mtaz/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/m3-mumtaz/
l/week9-new/Lecture/Lecture examples/5a_eventbinding_v0.py"
The process event fucntion was called  0
The process event fucntion was called  1
The process event fucntion was called  2
The process event fucntion was called  3
```

Event descriptors

UWE Bristol University of the West of England

Mouse-related event descriptors

Descriptor	Purpose of use
"<ButtonPress-1>”, "<ButtonPress-2>”, "<ButtonPress-3>"	Once clicked on mouse button<1,2,3>
"<B1-Motion>”, "<B2-Motion>”, "<B3-Motion>"	Changed mouse location while holding mouse button<1,2,3>
"<ButtonRelease-1>”, "<Double-Button-1>"	Once button<1>released, once button<1> double clicked
"<Enter>”, "<Leave>"	mouse pointer just moved/left over a particular widget.
"<FocusIn>”, "<FocusOut>"	A widget just received/lost the keyboard focus.
"<Configure>” …..	A widget just changed its size or position

Demo

Event descriptors

UWE Bristol University of the West of England

Keyboard-related event descriptors

Descriptor	Purpose of use
"<Key>”, "<Return>”, "<Backspace>"	Once pressed
"<Tab>”, "<Escape>”, "<Prior>”, "<Next>”,	Once pressed
"<Left>” “<Right>” "<Up>”, "<Down>"	Once pressed
"<F1>”, "<Shift>"	Once pressed

Demo

Read more: https://python-course.eu/tkinter/events-and-binds-in-tkinter.php

Event descriptors

An `event` object is passed to `event_handler` function with which relevant operations are organised via a number of attributes.

```
def process_event(event):
  if event.x > 10 and event.y > 20:
    # do something
```

Demo

Event Attributes

Event Object Attribute	Description
`.widget`	The widget this event was **bound** to. This is a reference to a `tkinter` widget instance; it is not a string name.
`.x, .y`	The current mouse position, relative to the application's window, in pixels.
`.x_root, .y_root`	The current mouse position relative to the upper left corner of the screen, in pixels.
`.char`	For keyboard events only, this is the character code of the key pressed or released as a string.
`.keysym`	For keyboard events only, the key symbol.
`.keycode`	For keyboard events only, the key code (i.e., the key's Unicode decimal value).
`.num`	For mouse button events only, the button number.
`.width, .height`	For configure events only, the new size of the widget, in pixels.

Roll Dice Example Demo

rollingdice.py → command
rollingdice_V1.py → multiple event binding

ASD > ASD > Lab8 > BB > Lecture examples > rollingdice.py > ...

```python
import random
import tkinter as tk

def roll():
    lbl_result["text"] = str(random.randint(1, 6))

window = tk.Tk()
window.columnconfigure(0, minsize=150)
window.rowconfigure([0, 1], minsize=50)

btn_roll = tk.Button(text="Roll", command=roll)
lbl_result = tk.Label()

btn_roll.grid(row=0, column=0, sticky="nsew")
lbl_result.grid(row=1, column=0)

window.mainloop()
```

Demo

ASD > ASD > Lab8 > BB > Lecture examples > rollingdice_V1.py >

```python
import random
import tkinter as tk

def roll(event):
    lbl_result["text"] = str(random.randint(1, 6))

window = tk.Tk()
window.columnconfigure(0, minsize=150)
window.rowconfigure([0, 1], minsize=50)

btn_roll = tk.Button(text="Roll")
btn_roll.bind("<Enter>", roll)
btn_roll.bind("<Return>", roll)
btn_roll.bind("<ButtonPress-1>", roll)
btn_roll.focus()

lbl_result = tk.Label()

btn_roll.grid(row=0, column=0, sticky="nsew")
lbl_result.grid(row=1, column=0)

window.mainloop()
```

#Example adapted from source: https://realpython.com/python-gui-tkinter/

Binding multiple handlers for same event

UWE Bristol University of the West of England

SD > ASD > Lab8 > BB > Lecture examples > 5g_multiplehandlersforsameevent.py > ...

```python
import tkinter as tk
from tkinter import ttk

def return_pressed(event):
    print('Return key pressed.')    #message on terminal

def log(event):
    print(event)    #displays full event details including x, y position and keycode

root = tk.Tk()

btn = ttk.Button(root, text='Save')
btn.bind('<Return>', return_pressed)    #<Return> is for Enter key
btn.bind('<Return>', log, add='+')  #Same event but multiple handlers require add parameter with '+'

btn.focus()
btn.pack(expand=True)

root.mainloop()
```

Handler

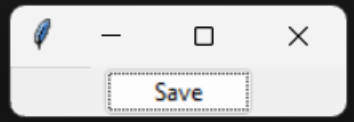
Event binding

Handler

Event binding

Demo

```
1/week9-new/Lecture/Lecture examples/5g_multiplehandlersforsameevent.py"
Return key pressed.
<KeyPress event send_event=True state=Mod1 keysym=Return keycode=13 char='\r' x=148 y=25>
Return key pressed.
<KeyPress event send_event=True state=Mod1 keysym=Return keycode=13 char='\r' x=-24 y=-5>
```



#Adapted from: https://www.pythontutorial.net/tkinter/tkinter-event-binding/

Staff Details Example

```python
from tkinter import *
import json

window=Tk()
window.title("Staff Members")

lstName=Listbox(window,width=100,height=100)
yscroll=Scrollbar(command=lstName.yview)
yscroll.grid(row=0,column=3,sticky=NS,rowspan=3)

lstName=Listbox(window,width=15,height=10,yscrollcommand=yscroll.set)
lstName.grid(row=0,padx=0,pady=0,sticky=W,columnspan=2,rowspan=3)

yscroll.config(command=lstName.yview)

txt = Text(window,width=30,height=10)
txt.grid(padx=5,pady=0,row=0,column=5)
```

Listbox

Scrollbar

Text

```python
with open("users.json",'r') as l:
    loaded_list = json.load(l)
    for line in loaded_list:
        countryName=line[0]
        lstName.insert('end',countryName)

def select(evt):
    value = lstName.curselection()[0]
    templist = loaded_list[value]

    txt.delete(1.0,END)
    for i in templist:
        txt.insert(END,i+'\n')

lstName.bind('<<ListboxSelect>>',select)

window.mainloop()
```

File handling

JSON object

Event binding

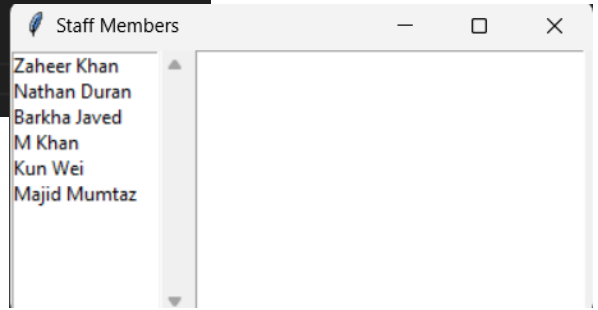
Demo

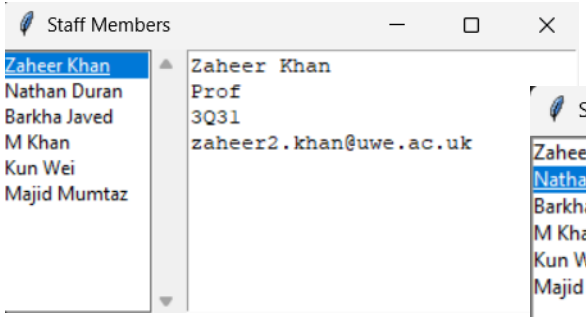
Staff Details Example

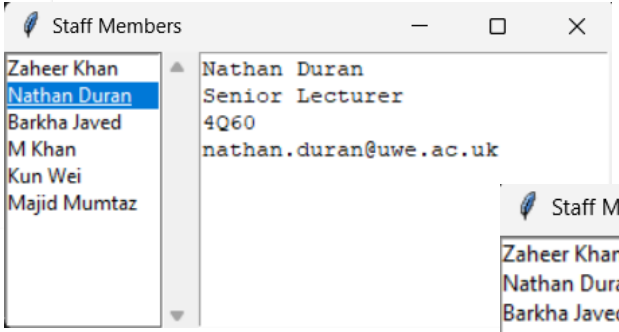
users.json > ...

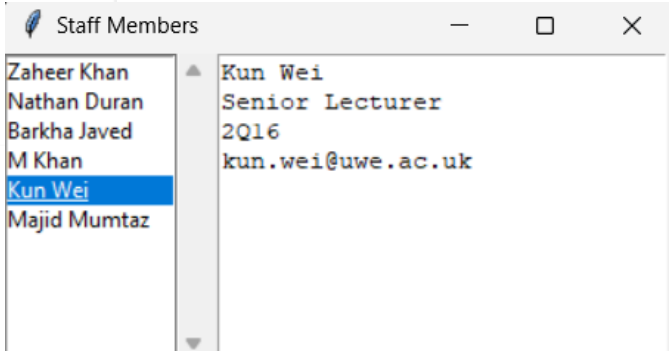
```
[
    ["Zaheer Khan", "Prof", "3Q31", "zaheer2.khan@uwe.ac.uk"], ["Nathan Duran", "Senior Lecturer", "4Q60", "nathan.duran@uwe.ac.uk"], ["Barkha Javed", "Lecturer", "3E40", "barkha.javed@uwe.ac.uk"], ["M Khan", "Senior Lecturer", "4Q58", "mkhan@uwe.ac.uk"], ["Kun Wei", "Senior Lecturer", "2Q16", "kun.wei@uwe.ac.uk"], ["Majid Mumtaz", "Sinor Lecturer", "4Q58", "majid.mumtaz@uwe.ac.uk"]

]
```

Json file





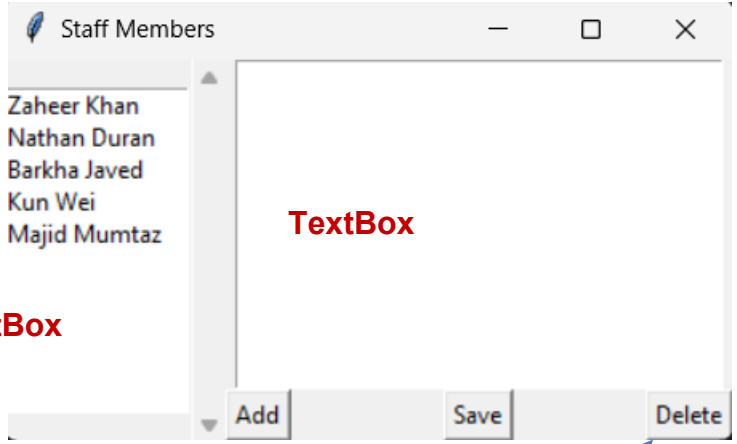


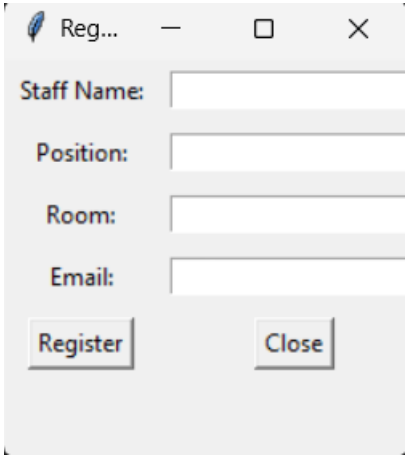


Homework:

Your practical exercise will be to add Menu Bar

Staff Details Example (Extended w/o OOP)





Add New Staff Registration

Add 3 Button Widgets

Demo Examples

1) 6a_Satff_v1_procedural.py

2) 6b_Satff_v1_OOP.py

Demo

Staff Details Example (Extended to MVC)

UWE Bristol University of the West of England

Model

MVC StaffApp Example

- Create Model
- Create View
- Create Controller
- Create mainApp (Optional)

5c_Staff_v4_MVC_Example > model.py > StaffModel > load_data

```python
import json

class StaffModel:
    def __init__(self):
        self.loaded_list = self.load_data()

    def load_data(self):
        try:
            with open("users.json", 'r') as f:
                return json.load(f)
        except FileNotFoundError:
            return []

    def save_data(self, data):
        with open("users.json", 'w') as f:
            json.dump(data, f)

    def add_staff(self, staff_data):
        self.loaded_list.append(staff_data)
        self.save_data(self.loaded_list)

    def delete_staff(self, index):
        if 0 <= index < len(self.loaded_list):
            del self.loaded_list[index]
            self.save_data(self.loaded_list)

    def get_staff_list(self):
        return self.loaded_list
```

Staff Details Example (Extended MVC)

View

MVC StaffApp Example

- Create Model
- Create View
- Create Controller
- Create mainApp (Optional)

6c_Staff_v4_MVC_Example > view.py > StaffView > __init__

```python
import tkinter as tk
from tkinter.messagebox import askyesno, showinfo

class StaffView:
    def __init__(self, root, controller):
        self.root = root
        self.controller = controller
        self.root.title("Staff Members")
        self.root.geometry("+850+250")

        self.uName = tk.StringVar()
        self.uEmail = tk.StringVar()
        self.uRoom = tk.StringVar()
        self.uPos = tk.StringVar()

        self.listName = tk.Listbox(self.root, width=15, height=10)
        self.yscroll = tk.Scrollbar(self.root, command=self.listName.yview)
        self.yscroll.grid(row=0, column=3, sticky=tk.NS, rowspan=3)

        self.listName.config(yscrollcommand=self.yscroll.set)
        self.listName.grid(row=0, padx=0, pady=0, sticky=tk.W, columnspan=2, rowspan=3)

        self.txt = tk.Text(self.root, width=30, height=10)
        self.txt.grid(padx=5, pady=0, row=0, column=5)

        self.frame = tk.Frame(self.root)
        self.frame.grid(row=1, column=5)
```

Staff Details Example (Extended MVC)

UWE Bristol University of the West of England

Controller

MVC StaffApp Example

- Create Model
- Create View
- Create Controller
- Create mainApp (Optional)

6c_Staff_v4_MVC_Example > controller.py > StaffController

```python
from view import StaffView

class StaffController:
    def __init__(self, root):
        self.model = StaffModel()
        self.view = StaffView(root, self)
        self.view.populate_listbox(self.model.get_staff_list())

    def add_new(self):
        self.view.show_add_staff_window()

    def register(self):
        staff_data = self.view.get_staff_data()
        self.model.add_staff(staff_data)
        self.view.populate_listbox(self.model.get_staff_list())
        self.view.show_info("Success", f"Staff '{staff_data[0]}' has been successfully registered.")
        self.view.clear_entries()

    def delete(self):
        try:
            index = self.view.listName.curselection()[0]
        except IndexError:
            self.view.show_info("Error", "Please select an item to delete.")
            return

        staff = self.model.get_staff_list()[index]
        if self.view.ask_yes_no("Confirm Deletion", f"Are you sure you want to delete '{staff[0]}'?"):
            self.model.delete_staff(index)
            self.view.populate_listbox(self.model.get_staff_list())
            self.view.show_staff_details([])
```

Staff Details Example (Extended MVC)

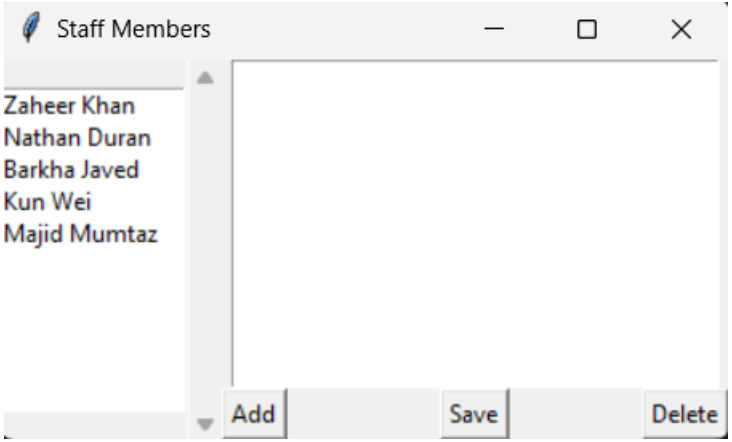
mainApp

MVC StaffApp Example

- Create Model
- Create View
- Create Controller
- Create mainApp (Optional)

6c_Staff_v4_MVC_Example > main.py > ...

```python
import tkinter as tk
from controller import StaffController

if __name__ == "__main__":
    root = tk.Tk()
    app = StaffController(root)
    root.mainloop()
```



Another Example – Tabbed windows and tabular data

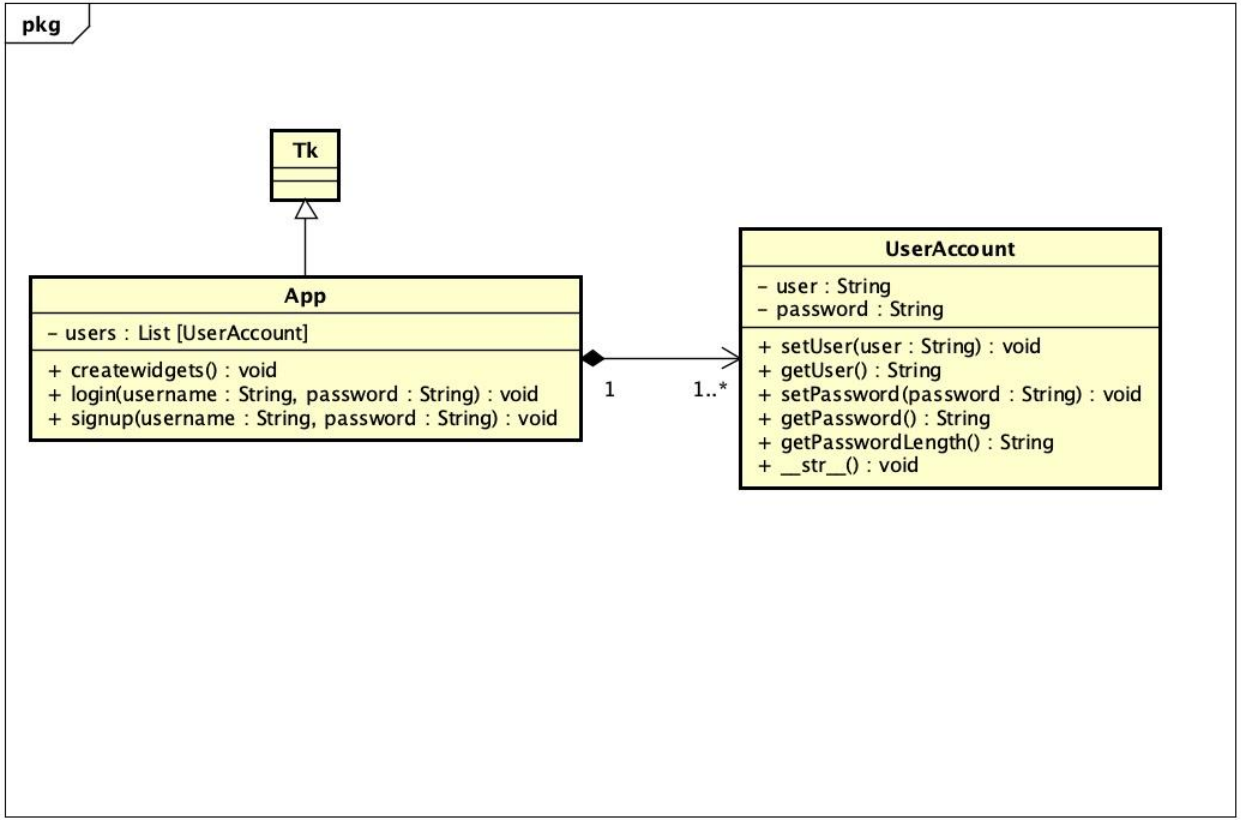
UWE Bristol University of the West of England

Demo Other examples e.g.,

Treeviewwidget_OOP.py,
Notebookwidget.py,
treeviewandnotebookwidget_v1.py

Homework

GUI with Tkinter – Login Example - OOP



Homework
By using
MVC based LoginApp

Implementation and Demo loginexamplehomework.py

References

- Official Documentation: https://docs.python.org/3/library/tkinter.html
- TKinter Tutorial 1: https://www.pythontutorial.net/tkinter/
- TKinter Tutorial 2: https://www.javatpoint.com/python-tkinter

Runstone Academy Tutorial
RIP Tutorial
RIP eBook

Summary and any questions ???

There are quite a good number of widgets that can help to build nice GUIs

You need to spend good time to practice widget

There are more Widget examples provided

Design your Interface of portfolio components.